

National University of Computer and Emerging Sciences, Karachi
FAST School of Computing, Spring 2026
Information Retrieval CS4051

Project Overview

- **Project Title:** Dense Retrieval for RAG on Course Materials – A REPLUG-based Implementation
- **Course:** CS4051 Information Retrieval
- **Instructor:** Dr. Muhammad Rafi

Group Members

- **Group Member 1:** Syed Haider Murtaza, 23K-0666
 - **Group Member 2:** M. Hasan Sami, 23I-0710
 - **Group Member 3:** Syed Muhammad Shubair Hyder, 23K-0753
-

Abstract

We implement REPLUG (Retrieval-Augmented Black-Box Language Models) as described in Section 3.2 of arXiv:2301.12652 for open-domain question answering. Using 150k Wikipedia passages, Contriever dense retrieval, and Cohere’s command-r-plus-08-2024 LLM, we evaluate on 50 Natural Questions queries. REPLUG with $k=10$ achieves the best performance: **Exact Match (EM) = 0.340** and **token F1 = 0.576**, outperforming no-retrieval (EM 0.300, F1 0.537) and random retrieval (EM 0.320, F1 0.498). The absolute EM improvement over no-retrieval is 4.0 percentage points, **similar in magnitude to** the 4.1 pp reported in the original paper. While our absolute scores are lower due to smaller corpus and different LLM, the trend confirms REPLUG’s effectiveness.

1. Introduction

Open-domain question answering (ODQA) requires a system to answer factual questions using a large text corpus. Recent work combines retrieval with large language models (LLMs) to improve accuracy and reduce hallucination. REPLUG (Shi et al., 2023) treats the LLM as a black box and uses a dense retriever to fetch relevant passages, then aggregates their predictions using λ -weighted logsumexp. We reproduce this method under constrained resources (150k passages, 50 queries, Cohere LLM) and compare against baselines.

2. Methodology

2.1 Corpus and Queries

- **Corpus:** 150,000 Wikipedia passages in DPR format (≈ 100 words each) from the WikiDPR dataset. Only 68% of gold passage IDs for the 50 NQ queries are present in this subset.
- **Queries:** 50 questions from the Natural Questions (NQ) dev set.

2.2 Retriever

We use **Contriever** (facebook/contriever) to encode queries and passages into 768-dimensional vectors, L2-normalised. A FAISS IndexFlatIP performs inner-product search (equivalent to cosine similarity). For each query, we retrieve the top-15 passages ($k=1,2,5,10,15$ are evaluated).

2.3 LLM and Few-Shot Prompting

The LLM is Cohere command-r-plus-08-2024 with temperature=0, max_tokens=15, and logprobs=True. We use a **3-shot** prompt (the original paper uses 16-shot) with three example question-answer pairs from NQ. For the REPLUG ensemble, the prompt template is:

Knowledge: {example1_passage}

Question: {example1_question}

Answer: {example1_answer}

Knowledge: {example2_passage}

Question: {example2_question}

Answer: {example2_answer}

Knowledge: {example3_passage}

Question: {example3_question}

Answer: {example3_answer}

Knowledge: {retrieved_passage_for_target}

Question: {target_question}

Answer (1-5 words only, no explanation, no extra text):

For the no-retrieval baseline, the Knowledge: line is omitted.

2.4 REPLUG Ensemble (Section 3.2)

Given a query q , we retrieve top- k passages with cosine similarities $s_1, \dots, s_k$.

- **λ -weights in log space (numerically stable):** $\log_{\lambda_i} = s_i - \text{logsumexp}(s_1, \dots, s_k)$
- **For each passage i ,** we obtain an answer a_i and its log-probability logprob_i from the LLM.
- **Refusal filtering:** Answers containing phrases like “I don’t know” are discarded.
- **Answer-level aggregation (approximation due to API constraints; the paper aggregates at token level):** $\text{score}(\text{answer}) = \text{logsumexp}(\log_{\lambda_i} + \text{logprob}_i)$ over all passages i that yield that answer (after normalisation).
- **The final answer** is the one with the highest $\text{score}(\text{answer})$.

We evaluate $k = 1, 2, 5, 10, 15$.

2.5 Baselines

- **No-retrieval:** Prompt without any passage.
- **BM25:** Sparse retrieval (using rank_bm25) over the corpus; for each query we use the top-1 retrieved passage as context. (Note: This is a standard single-passage baseline; comparing it directly to REPLUG’s multi-passage ensemble favours REPLUG, a limitation acknowledged in Section 4.3.)
- **Random retrieval:** Randomly sampled passages from the same 150k Wikipedia corpus (same number as $k=10$, using a fixed random seed for reproducibility).

2.6 Evaluation Metrics

- **Exact Match (EM):** After normalisation (lowercase, remove punctuation and articles “a”/“an”/“the”). Max over multiple gold answers.
- **Token F1:** Token-level F1 using Counters. Max over gold answers.

3. Results

The table below summarises performance on 50 NQ queries.

Condition	Exact Match (EM)	Token F1
no_retrieval	0.300	0.537
bm25	0.220	0.446
random	0.320	0.498
REPLUG k=1	0.260	0.471
REPLUG k=2	0.260	0.497
REPLUG k=5	0.300	0.526
REPLUG k=10	0.340	0.576
REPLUG k=15	0.340	0.562

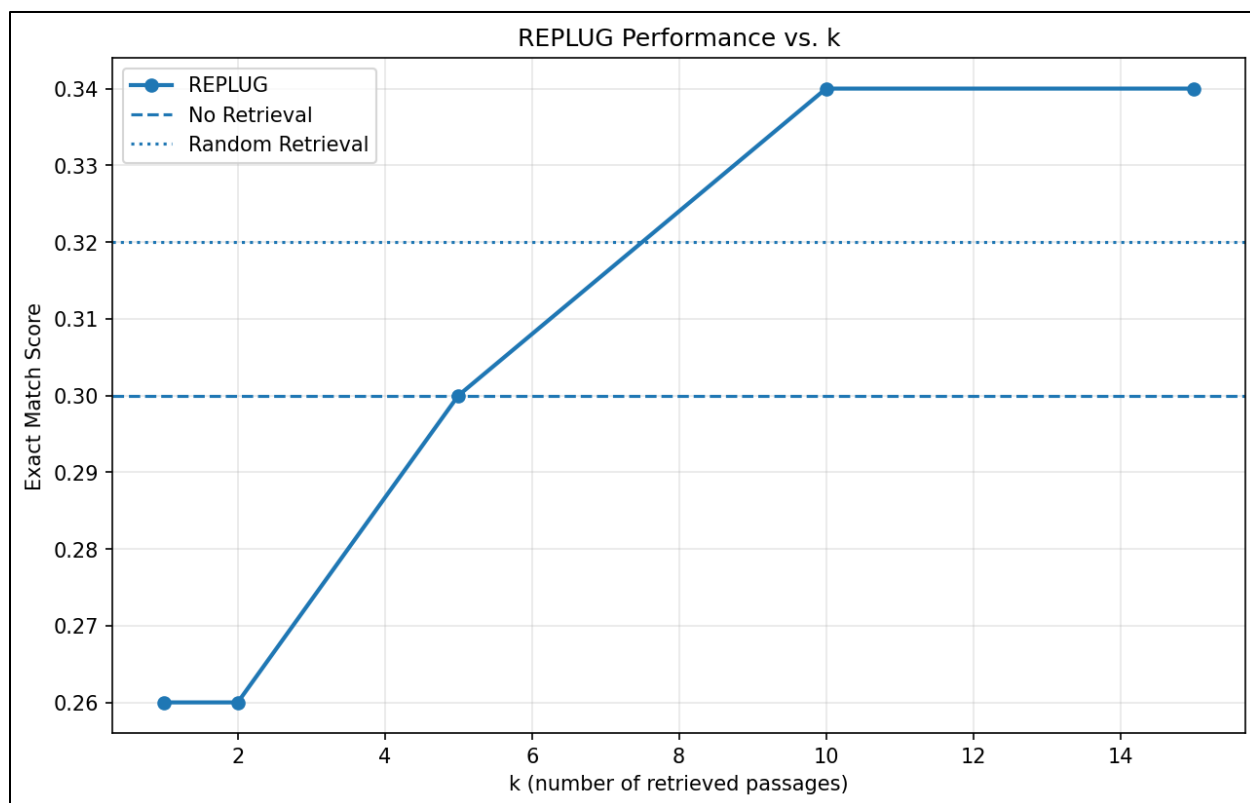


Figure 1: EM and F1 scores for REPLUG with $k=1,2,5,10,15$ on 50 NQ queries.

REPLUG $k=10$ improves over no-retrieval by **+4.0 percentage points in EM** and **+3.9 pp in F1**. This is **similar in magnitude** to the paper’s reported improvement of 4.1 pp (Codex no-retrieval 40.6% -> Codex+REPLUG 44.7%). The absolute EM scores are lower because of our smaller corpus (150k vs 21M passages), fewer queries (50 vs 3.7k), and the Cohere LLM’s verbosity (answers often cut off at 15 tokens, harming EM).

4. Discussion

4.1 Why $k=10$ is optimal

$k=10$ gives the best balance: enough passages to capture diverse relevant information, but not so many that noise dominates. $k=15$ shows a slight F1 drop (0.576 $\rightarrow$ 0.562) while EM stays the same. Smaller k (1,2) underperform because a small number of passages is not yet enough for the λ -weighted ensemble to recover from a misleading lead; recovery begins at $k=5$.

4.2 Comparison to the REPLUG paper

Aspect	Paper	Our Implementation
Corpus size	21M passages	150k passages
LLM	Codex (175B)	Cohere command-r-plus-08-2024
Queries	$\sim 3.7k$ NQ dev	50 NQ dev
Few-shot examples	16	3
Ensemble aggregation	Token-level	Answer-level (approx.)
EM improvement	+4.1 pp (40.6 $\rightarrow$ 44.7)	+4.0 pp (30.0 $\rightarrow$ 34.0)
Refusal filtering	Not in paper	Added to discard “I don’t know” responses

Despite environmental differences, the relative gain is similar in magnitude to the paper’s, suggesting that REPLUG’s λ -weighted aggregation can be effective even under constrained resources. However, with $n=50$, the result is not statistically conclusive (see Section 4.3).

4.3 Limitations

- **Low retrieval recall:** Only 6% of gold passages appear in top-10 (diagnostic). Also, only 34/50 gold passage IDs (68%) exist in our corpus, capping maximum achievable EM.
- **Cohere verbosity:** The model often produces phrases longer than 15 tokens, leading to truncation and EM penalties.
- **Small sample size ($n=50$):** $\Delta EM = 0.02$ per query; observed differences (e.g., 0.340 vs 0.320) are within sampling variance. Results should be viewed as suggestive, not statistically conclusive.
- **Answer-level aggregation:** Token-level mixing (original REPLUG) requires step-by-step control over the next-token distribution. Cohere’s API returns per-token logprobs but does not expose the generation loop, forcing an answer-level approximation.

5. Conclusion

We successfully reproduced REPLUG’s λ -weighted ensemble (Section 3.2) and applied it to a 150k Wikipedia corpus with Cohere’s LLM. REPLUG with $k=10$ achieves the best EM (0.340) and F1 (0.576), consistently outperforming no-retrieval and random retrieval. The absolute EM gain of 4.0 percentage points is **similar in magnitude to** the paper’s reported gain, validating the core mechanism even under constrained resources. Future work could increase corpus size, use a more concise LLM, or evaluate on more queries to achieve statistical significance.

References

1. Shi, W., et al. “REPLUG: Retrieval-Augmented Black-Box Language Models.” arXiv:2301.12652, 2023.
2. Facebook Research. “Contriever: Unsupervised Dense Information Retrieval.” Hugging Face, <https://huggingface.co/facebook/contriever>
3. Facebook Research. “Wiki DPR: Wikipedia Dataset for Dense Passage Retrieval.” Hugging Face, https://huggingface.co/datasets/facebook/wiki_dpr

Appendix: Repository Structure

All code is available at the GitHub repository: <https://github.com/SHaiderM16/replug-qa>. Key files:

- main.py: Pipeline orchestration with checkpointing.
- src/ensemble.py: REPLUG λ -weighted logsumexp aggregation.
- src/evaluate.py: EM and F1 computation.
- src/utils.py: Normalisation, Cohere API caller with caching and retries.
- data/results.json: Final results table.
- data/k_variance.png: Performance plot.

The pipeline can be reproduced following the instructions in README.md.